

Computer Systems

- BA4

1 Lecture 5 - Network & File Syscalls

Permission bits

Am I the **owner**? → use USR bits.
Am I in the **group**? → use GRP bits.
Otherwise → use OTH bits.
Only the first matching category applies — even if another gives more rights.

Syscall success conditions

<code>open(O_RDONLY)</code>	<code>r</code> on file
<code>open(O_WRONLY)</code>	<code>w</code> on file
<code>open(O_RDWR)</code>	<code>r+w</code> on file
<code>open(O_CREAT)</code>	<code>w+x</code> on dir
<code>read(fd,...)</code>	<code>fd</code> open, mode=read
<code>write(fd,...)</code>	<code>fd</code> open, mode=write
<code>close(fd)</code>	<code>fd</code> open

Every dir in the path needs `x` to be traversed.
Any syscall on a **closed** `fd` fails immediately.

<code>S_*</code> flags
Only used with <code>O_CREAT</code> to set the new file's permissions. Ignored otherwise.

2 Lecture 8 - Forwarding and IP
Forwarded using the longest matching prefix .

Broadcast IP address (+1)
The biggest IP address an IP subnet. If a packet has this destination, it's destined to all the end-systems and routers in the IP subnet.

Global reachability
Every end-system must be reachable from any other end-system.

NAT: Layering violation (but it works)
A router becomes a NAT only when it's configured to translate between private and public addresses. - Requires per-connection state at the border routers of the private IP subnets. - An end-system inside a private IP subnet is un-reachable from the outside world. [trick possible]

3 Lecture 9 - Routing & BGP

Dijkstra's (Link state: converges faster)
1. At each step, consider a new router starting from "closest" neighbor 2. Check whether current paths can be improved 3. End when no improvement is possible

Bellman-Ford (Distance vector: less bandwithh)
1. All neighbors exchange information 2. Each router checks whether it can improve current paths by leveraging the new information 3. End when no improvement is possible

Internet routing challenges
Admin: An ISP may want to hide its link costs Scale: Too many IP subnets may cause trouble

Border Gateway Protocol (BGP)
Inter-AS Routing : Not always the SP. Bellman 1. All neighbors exchange reachable prefixes 2. Each router checks whether it can improve paths 3. End when no improvement is possible

Internet routing solutions
- Router ↔ with local router - Border router ↔ with external neighbor - Router keeps forwarding state for (1) local IP subnets and (2) foreign aggregate IP prefixe

4 Lecture 10 - Link layer & Ethernet

Address Resolution Protocol (ARP)
Resolves an IP address to a MAC address (to build the L2 frame) via <i>broadcast</i> + <i>caching</i> . Naïve broadcast may cause forwarding loops ⇒ use a spanning tree to eliminate cycles.

5 Lecture 11 - File Systems

Persistence
State that outlives the process that created it (unlike DRAM, which is volatile).

File System
Abstracts persistent blocks into: Name, Organize, Share, Protect . Built on files and directories .

File
Named array of bytes (untyped) with data + meta-data (size, owner, permissions, ...).

Inode
Fixed-size on-disk record — one per file: metadata + pointers to data blocks .

Inode Table
Fixed array indexed by inode number.

Directory (special file)
Array of { filename , inode } pairs per partition

File Descriptor Table
Per-process. <code>fd</code> → (inode , mode , offset). 0=STDIN , 1=STDOUT , 2=STDERR .

6 Lecture 12 - Processes and Threads

Process
An instance of a running program. Has its own isolated memory space (code, data, heap, stack) and OS resources (file descriptors, etc.).

Thread
A unit of execution within a process. Threads share the process's memory and resources, but each has its own stack and program counter .

Key properties
Identifier: unique id for processes and threads - Each Thread/Process has the illusion that is it the only one occupying the CPU/Memory (resp.)

7 Lecture 13 - Sharing the CPU

CPU mode & instructions
CPU stores a privilege level: Kernel mode (high) or User mode (low). Non-privileged: executable anytime Privileged: only in kernel mode

Limited direct execution (invoking the kernel)
Syscall / Exception / Interrupt

Process lifecycle
fork: creates a child process (copy of parent) exec: replaces process image with a new program exit: terminates the process wait: parent waits for child to finish

Type of system processes
Init: first process after boot Idle: runs when nothing else can Shell: spawns new process on command GUI manager: spawns new process on click

8 Lecture 14 - Memory & Loose Ends

End of a process
<code>exit()</code> syscall: kernel destroys memory image but keeps exit status ⇒ process becomes a zombie until parent reaps it via <code>wait()</code> to read the exit status.

9 Lecture 15 - CPU Scheduling

Scheduling Metrics
Turnaround time = completion - arrival. Response time = first run - arrival.

FIFO
Run jobs in arrival order. Suffers from convoy effect : short jobs stuck behind long ones.

SJF (Shortest Job First)
Run shortest job first. Optimal turnaround if all jobs arrive simultaneously. Non-preemptive.

STCF (Shortest Time-to-Completion First)
Preemptive SJF: preempt if a new job has less remaining time. Optimal turnaround.

Round Robin (RR)
Each job runs for a fixed quantum , then preempted. Good response time, poor turnaround.

Tradeoff
SJF/STCF optimizes turnaround . RR optimizes response time . Hard to do both.

MLFQ (Multi-Level Feedback Queue)
Multiple queues; lower priority = longer time slice. R1: priority(A) > priority(B) ⇒ A runs R2: priority(A) = priority(B) ⇒ RR R3: new job starts at top priority R4: uses full slice ⇒ demoted; blocks early ⇒ keeps priority R5: periodically move all jobs to top (priority boost) — avoids starvation

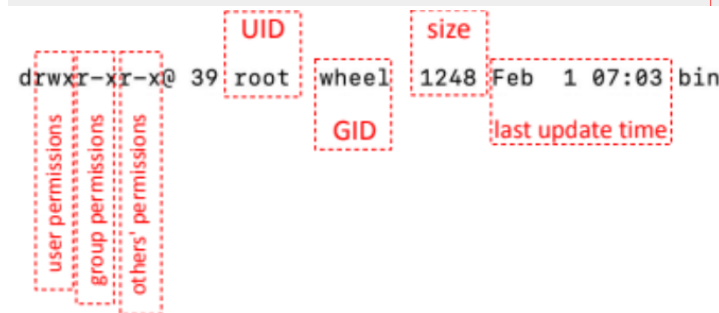
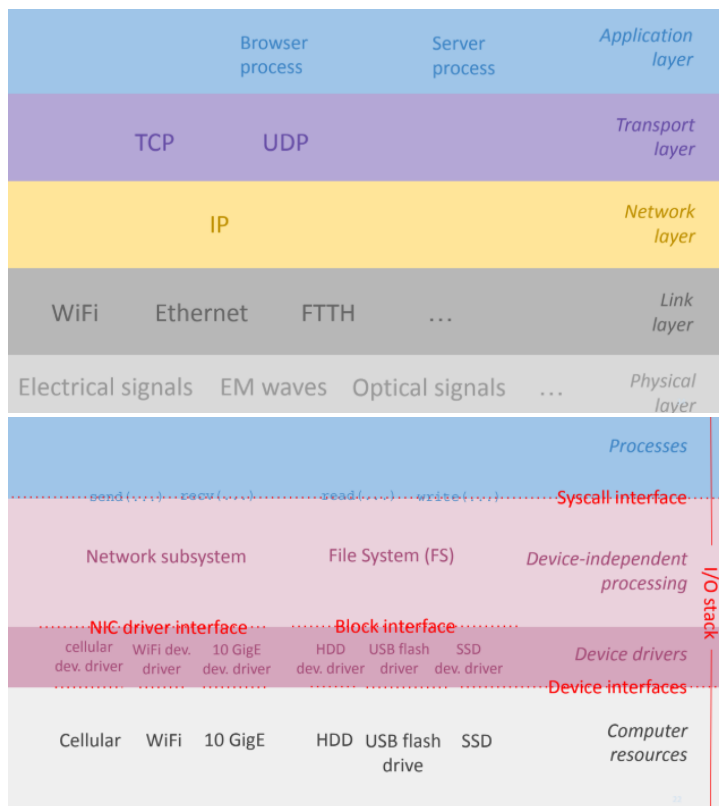
10 Lecture 17/18 - Concurrency

Concurrency: how should we manage tasks Parallism: how to execute tasks in parallel
--

Race condition & Data race
Race cond.: behavior depends on thread timing Data race: unsynchronized concurrent access to shared writable state

Critical section
Code that will be executed by the threads. Must enforce: atomicity (uninterruptible) and mutual exclusion (one thread at a time).

Synchronization primitives
Lock → prevent concurrent access Condition var. → wait until a condition holds Semaphore → count available resources



open() flags / permission mode	
O_CREAT	create file if it doesn't exist
O_EXCL	fail if file already exists
O_APPEND	always write at end of file
O_TRUNC	truncate file on open
O_RDONLY	read only
O_WRONLY	write only
S_IRUSR	read by owner
S_IWUSR	write by owner
S_IXUSR	execute by owner
S_IRGRP	read by group
S_IROTH	read by others
0	no permissions

```

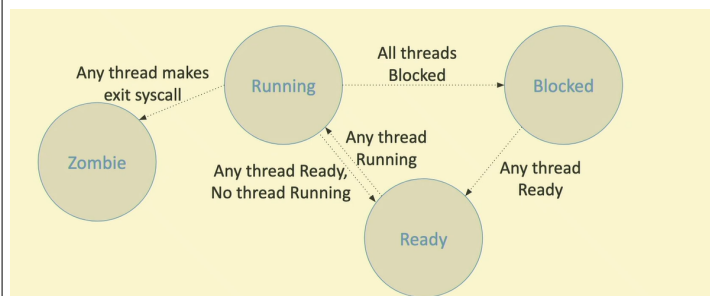
INIT-SINGLE-SOURCE(G, s)
  s.d = 0
  for v in G.V: v.d = inf; v.pi = NIL

RELAX(u, v, w)
  if v.d > u.d + w(u, v)
    v.d = u.d + w(u, v); v.pi = u

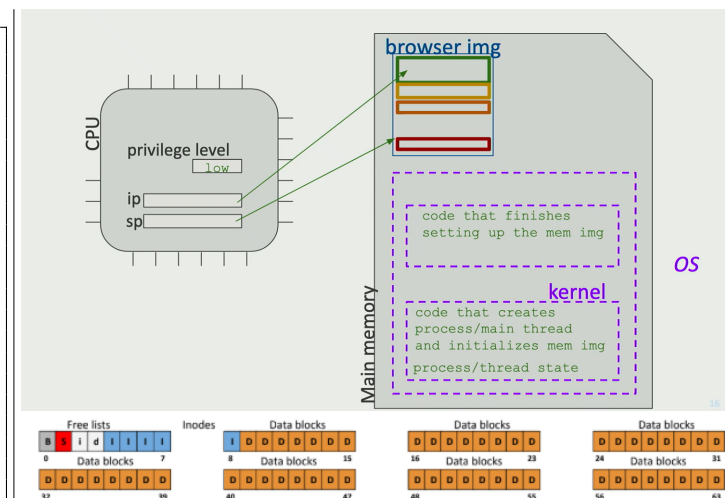
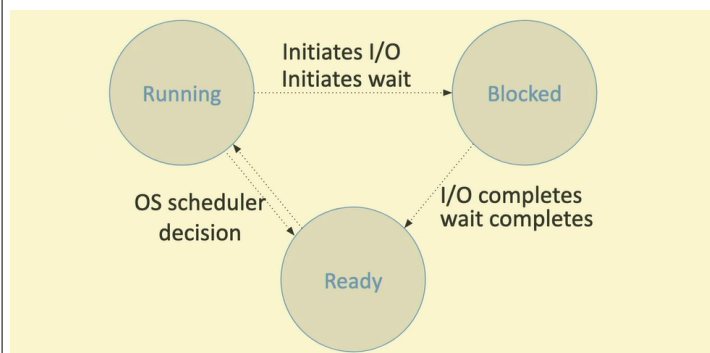
BELLMAN-FORD(G, w, s)
  INIT-SINGLE-SOURCE(G, s)
  for i = 1 to |G.V| - 1
    for (u, v) in G.E:
      RELAX(u, v, w)

DIJKSTRA(G, w, s)
  INIT-SINGLE-SOURCE(G, s)
  S = {}
  Q = G.V // min-priority queue (sorted by d(v))
  while Q != {}:
    u = EXTRACT-MIN(Q)
    S = S U {u}
    for each vertex v in G.Adj[u]:
      RELAX(u, v, w)
  
```

Process state diagram



Thread state diagram



Powers of ten

- 1 KB (kilobyte) = $10^3 = 1\,000$ bytes
- 1 MB (megabyte) = $10^6 = 1\,000\,000$ bytes
- 1 GB (gigabyte) = $10^9 = 1\,000\,000\,000$ bytes

Powers of two

- 1 KiB (kibibyte) = $2^{10} = 1\,024$ bytes
- 1 MiB (mebibyte) = $2^{20} = 1\,048\,576$ bytes
- 1 GiB (gibibyte) = $2^{30} = 1\,073\,741\,824$ bytes